

***BRAC*'s**

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 4

Roll no: 28 PRN No: 12411933 Name of Student: **Pragati S. Rakhunde**

Subject Name : Deep Learning

Problem Statement :

Develop an LSTM-based model for time-series forecasting using a stock price dataset. The model should use historical closing prices to learn temporal patterns and predict future stock prices. Evaluate the forecasting performance using appropriate regression metrics and visualize the actual and predicted values.

Objective :

- To understand the use of LSTM networks for time-series forecasting.
- To preprocess and normalize historical stock-price data.
- To create sequential input windows from historical closing prices.
- To train an LSTM model and predict future closing prices.
- To evaluate the model using MAE, RMSE and MAPE.

Dataset :

Stock market historical data for AAPL (Apple Inc.) is used. The implementation downloads approximately 5 years of daily historical data using yfinance. The closing-price column is used as the main forecasting variable. If a local stock_data.csv file is provided, the program uses its Close column instead.

Algorithm :

1. Import the required libraries such as NumPy, Pandas, Matplotlib, Scikit-learn and TensorFlow/Keras.
2. Load the historical stock-price dataset and select the Close price.
3. Split the data into training and testing sets using an 80:20 ratio.
4. Fit MinMaxScaler only on the training data and transform both training and testing data.
5. Create sequences using a look-back window of 60 days; the previous 60 closing prices are used to predict the next closing price.
6. Reshape the input into the LSTM format: samples $\times$ time steps $\times$ features.
7. Build the LSTM model with two LSTM layers, Dropout layers and a Dense output layer.

8. Compile the model using the Adam optimizer and mean squared error loss.
9. Train the model for 20 epochs with a batch size of 32 and validation data.
10. Generate predictions on the test sequences and inverse-transform them to the original price scale.
11. Calculate MAE, RMSE and MAPE to measure forecasting performance.
12. Plot training/validation loss and actual versus predicted stock prices.
13. Save the trained model as lstm_stock_model.keras.

Code :

The complete implementation is provided in the accompanying Python file lstm_stock_forecasting.py. The main model architecture is:

```
model = Sequential([
    LSTM(50, return_sequences=True, input_shape=(LOOKBACK, 1)),
    Dropout(0.2),
    LSTM(50),
    Dropout(0.2),
    Dense(1)
])

model.compile(optimizer="adam",
              loss="mean_squared_error")
```

Output :

Insert the following screenshots after executing the program:

Model Summary / Training

```
Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(**kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 50)	10,400
dropout (Dropout)	(None, 60, 50)	0
lstm_1 (LSTM)	(None, 50)	20,200
dropout_1 (Dropout)	(None, 50)	0
dense (Dense)	(None, 1)	51

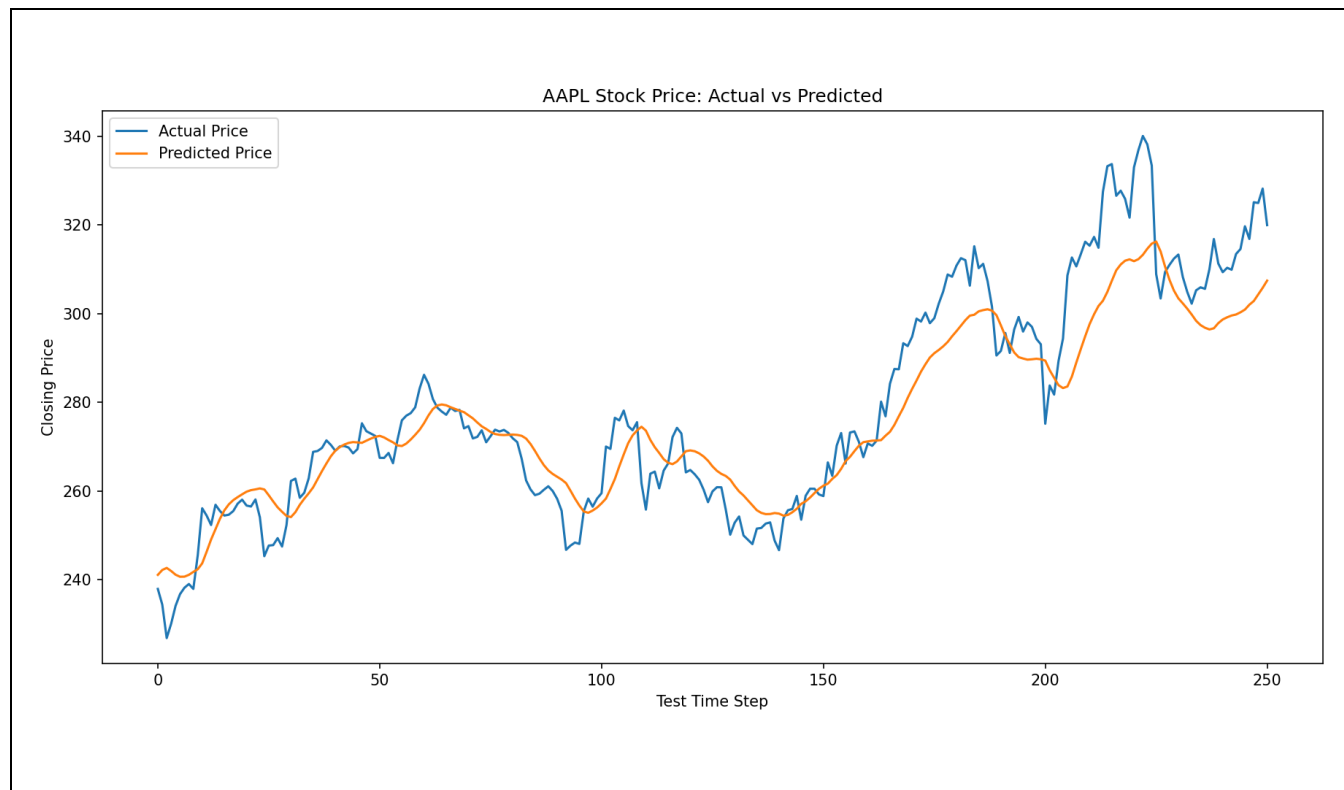
```
Total params: 30,651 (119.73 KB)
Trainable params: 30,651 (119.73 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/20
27/27 ██████████ 18s 171ms/step - loss: 0.0136 - val_loss: 0.0127
```

Evaluation Metrics

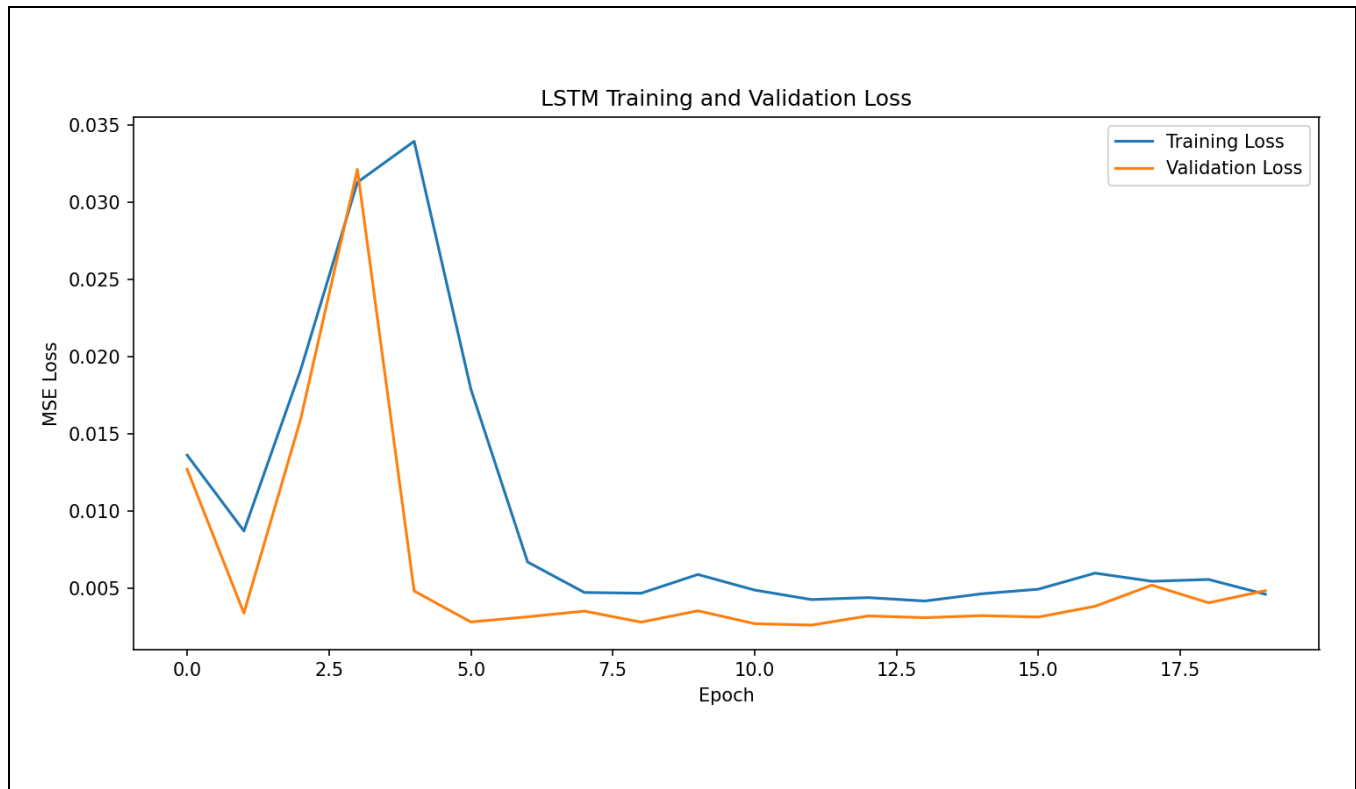
```
Epoch 19/20
27/27 ██████████ 3s 104ms/step - loss: 0.0056 - val_loss: 0.0041
Epoch 20/20
27/27 ██████████ 3s 102ms/step - loss: 0.0046 - val_loss: 0.0049

===== Evaluation Metrics =====
MAE   : 7.4609
RMSE  : 9.7076
MAPE  : 2.59%
```

Actual vs Predicted Graph



Training and Validation Loss



Evaluation Metrics :

Metric	Meaning	Purpose
MAE	Mean Absolute Error	Measures average absolute difference between actual and predicted prices.
RMSE	Root Mean Squared Error	Penalizes larger prediction errors more strongly.
MAPE	Mean Absolute Percentage Error	Expresses prediction error as a percentage of actual values.

Result :

The LSTM model learns temporal patterns from the historical stock closing-price sequence and produces predicted prices for the test period. The final MAE, RMSE and MAPE values obtained after execution should be recorded from the program output. The actual-versus-predicted plot can be used to visually compare forecasting performance.

Conclusion :

An LSTM-based time-series forecasting model was successfully implemented for stock-price prediction. The experiment demonstrates how sequential historical observations can be converted into fixed-length time windows and learned using LSTM layers. Model performance is evaluated quantitatively using MAE, RMSE and MAPE and visually using actual-versus-predicted plots.